

Important MATLAB Functions for the Flight Mechanics Course

Version 1.0

Buettelborn 24th August 2019

Dr. Danyck Nguewo

1. **poly** (Polynomial with specified roots or characteristic polynomial)

`p = poly(r)`

Where `r` is a vector, returns the coefficients of the polynomial whose roots are the elements of `r`.

`p = poly(A)`

Where `A` is an n -by- n matrix, returns the $n+1$ coefficients of the characteristic polynomial of the matrix, $\det(\lambda I - A)$.

2. **eig** (Eigenvalues and eigenvectors)

`e = eig(A)`

Returns a column vector containing the eigenvalues of square matrix `A`.

`[V,D] = eig(A)`

returns diagonal matrix `D` of eigenvalues and matrix `V` whose columns are the corresponding right eigenvectors, so that $A*V = V*D$.

3. **roots** (Polynomial roots)

`r = roots(p)`

Returns the roots of the polynomial represented by `p` as a column vector. Input `p` is a vector containing $n+1$ polynomial coefficients, starting with the coefficient of x^n . A coefficient of 0 indicates an intermediate power that is not present in the equation. For example, `p = [3 2 -2]` represents the polynomial $3x^2+2x-2$.

4. **ss** (Create state-space model, convert to state-space model)

`sys = ss(A,B,C,D)`

Use `ss` to create state-space models (`ss` model objects) with real- or complex-valued matrices or to convert dynamic system models to state-space model form. You can also use `ss` to create Generalized state-space (`genss`) models.

5. **tf** (Transfer function model)

`sys = ss(A,B,C,D), sys = tf(sys)`

Converts the dynamic system model `sys` to a transfer function model.

`TF = tf(numerator,denominator)`

Creates a continuous-time transfer function model, setting the Numerator and Denominator properties. For instance, consider a continuous-time SISO dynamic system represented by the transfer function $\text{sys}(s) = N(s)/D(s)$, the input arguments `numerator` and `denominator` are the coefficients of $N(s)$ and $D(s)$, respectively.

6. **ss2tf** (Convert state-space representation to transfer function)

`[b,a] = ss2tf(A,B,C,D)`

Converts a state-space representation of a system into an equivalent transfer function. `ss2tf` returns the Laplace-transform transfer function for continuous-time systems and the Z-transform transfer function for discrete-time systems.

7. **tfdata** (Access transfer function data)

`[num,den] = tfdata(sys)`

Returns the numerator(s) and denominator(s) of the transfer function for the TF, SS or ZPK model (or LTI array of TF, SS or ZPK models) `sys`. For single LTI models, the outputs `num` and `den` of `tfdata` are cell arrays with the following characteristics:

8. **ssdata** (Access state-space model data)

`[a,b,c,d] = ssdata(sys)`

Extracts the matrix (or multidimensional array) data `A`, `B`, `C`, `D` from the state-space model (LTI array) `sys`. If `sys` is a transfer function or zero-pole-gain model (LTI array), it is first converted to state space. See `ss` for more information on the format of state-space model data.

9. **imag** (Imaginary part of complex number)

`Y = imag(Z)`

Returns the imaginary part of each element in array `Z`.

10. **real** (Real part of complex number)

`X = real(Z)`

Returns the real part of each element in array `Z`.

11. **bodeplot** (Plot Bode frequency response with additional plot customization options)

`h = bodeplot(sys)`

Plot the Bode magnitude and phase of the dynamic system model `sys` and returns the plot handle `h` to the plot. You can use this handle to customize the plot with the `getoptions` and `setoptions` commands.

`bodeplot(..., plotoptions)`

Plots the Bode response with the options specified in `plotoptions`

12. **angle** (Phase angle)

$\theta = \text{angle}(z)$

Returns the phase angle in the interval $[-\pi, \pi]$ for each element of a complex array z . The angles in θ are such that $z = \text{abs}(z) \cdot \exp(i \cdot \theta)$.

13. **abs** (Absolute value and complex magnitude)

$Y = \text{abs}(X)$

Returns the absolute value of each element in array X .

If X is complex, $\text{abs}(X)$ returns the complex magnitude.